Musterhandel Agentic Voicebot in Genesys Cloud

Vereinfachte Architektur- und Implementierungsdokumentation

Hinweis: Diese Darstellung ist bewusst vereinfacht. Interne Herstellerarchitektur wurde aus Gründen des Unternehmensgeheimnis nicht geteilt.

Stand: 01.05.2026

Repository: https://github.com/georgyrudnev/Musterhandel-Agentic-Voicebot

1. 1. Executive Summary

Das Projekt ist kein klassisches Software-Backend mit eigenem Java-, Python- oder Node-Service. Der größte Teil der Logik lebt in der Genesys-Cloud-Konfiguration: im Architect-Bot-Flow, im Agentic Virtual Agent aus AI Studio sowie in angebundenen Datenquellen und Aktionen.

Das GitHub-Repository enthält vor allem Exportartefakte einer laufenden Genesys-Lösung. Diese Artefakte zeigen, wie Gesprächslogik, Agentenverhalten, Datenzugriff und Eskalation modelliert wurden.

> Das „Backend“ besteht hier hauptsächlich aus Plattform-Konfiguration und Integrationen innerhalb von Genesys Cloud.
>
> • Architect orchestriert den Gesprächsablauf und ruft den agentischen AI-Studio-Agenten auf.
>
> • Der Agent verwendet bei Bedarf Tools (Wissen, Lookup, Workitem-Anlage, Lieferstatus) und gibt bei Bedarf eine Eskalation zurück.

Genesys-Begriffe in sehr einfacher Form

Begriff	Bedeutung im Projekt
Architect	Grafischer Flow-Designer und Orchestrierungswerkzeug für die Voice-Interaktion.
AI Studio	Bereich in Genesys Cloud, in dem der Agentic Virtual Agent definiert und veröffentlicht wird.
Call Agentic Virtual Agent	Architect-Aktion, die den veröffentlichten AI-Studio-Agenten im Gespräch startet.
Data Action	Integrationsaufruf zu Daten, APIs oder Plattformfunktionen.
Data Table	Kleine strukturierte Lookup-Tabelle in Genesys Cloud, z. B. für PLZ -> Markt.
Knowledge	Wissensbasis für FAQ-/Produktantworten.
Workitem	Nicht-sprachlicher Arbeitsauftrag für manuelle Nachbearbeitung.

2. 2. Was im Repository enthalten ist

Das Repository bildet die Lösung nicht als Quellcode-Anwendung, sondern als versionierbare Konfiguration ab.

Datei	Genesys-Baustein	Rolle im Projekt
GR Testhandel AVA v2_v22-0.yaml	Architect Bot Flow Export	Enthält Flow-Struktur, Variablen, Intents/Slots, Task-Logik, Eskalation und den Aufruf des Agentic Virtual Agent.
gro-handel-markt-finder---registrierung-export.json	AI Studio Agent Export	Enthält Rolle, Guidelines, Guardrails, Tools, Ereignisse und die fachliche Logik des agentischen Agents.
Handelsmärkte PLZ Standort.csv	Data Table Import	Enthält 5.000 Zeilen für die Zuordnung von PLZ zu nächstem Markt (4 Spalten).

Wichtig: Die Exporte machen die Logik nachvollziehbar, aber sie sind nicht die komplette Laufzeitumgebung. In einer neuen Genesys-Organisation müssen Integrationen, Berechtigungen und bestimmte Zuordnungen erneut verdrahtet werden.

3. 3. Vereinfachte Architektur

Die folgende Grafik reduziert die Plattform bewusst auf die Komponenten, die im Projekt tatsächlich relevant sind. Sie ist daher keine offizielle Herstellerarchitektur, sondern eine abgabefähige Projektansicht.

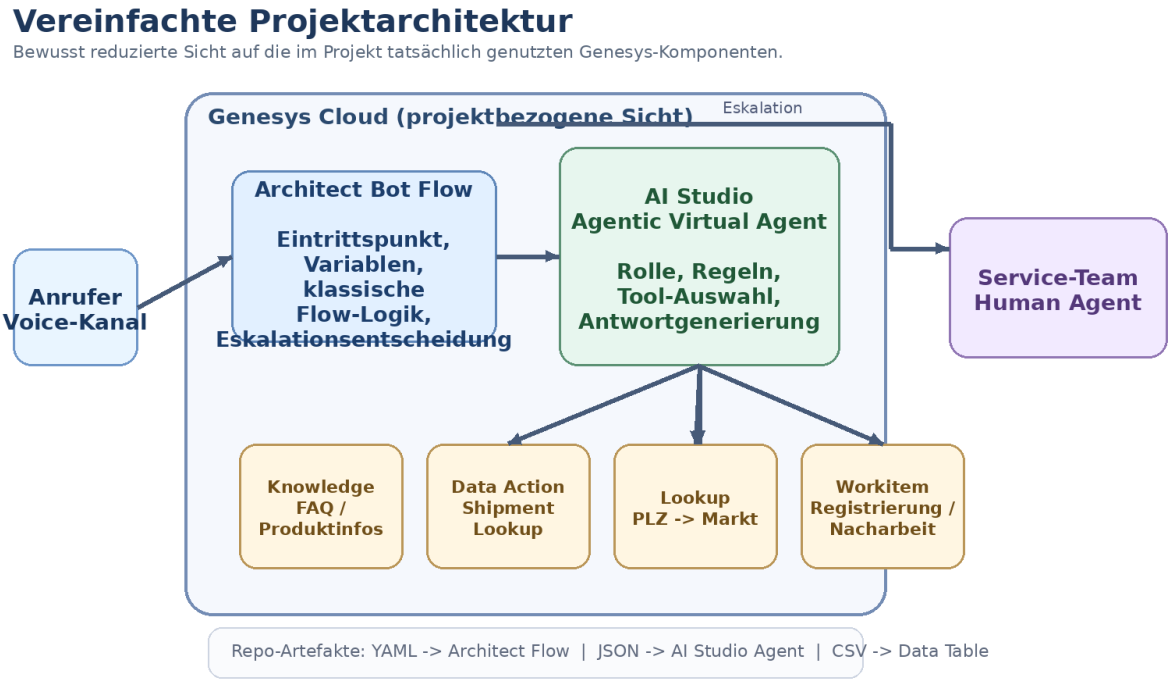

Abbildung 1: Vereinfachte Projektarchitektur

4. 4. Wie ein Gespräch technisch abläuft

Die Laufzeit lässt sich vereinfacht in sieben Schritte zerlegen:

Nr.	Komponente	Was passiert?
1	Voice / Anrufer	Ein Kunde ruft an und landet im Genesys-Voice-Kanal.
2	Architect	Der Bot Flow übernimmt die Interaktion, hält Variablen vor und ruft den Agentic Virtual Agent auf.
3	AI Studio Agent	Der Agent interpretiert das Anliegen anhand seiner Rolle, Regeln und Guardrails.
4	Tools / Wissen	Falls nötig ruft der Agent ein Tool auf: Produktwissen, Lieferstatus, Markt-Lookup oder Registrierungsprozess.
5	Tool-Ergebnis	Das Ergebnis kommt strukturiert zurück (z. B. Marktname, ETA oder Workitem-ID).
6	Agent -> Architect	Der Agent formuliert eine Antwort oder signalisiert eine Eskalation per Exit-Information.
7	Architect / Mensch	Architect beendet den Call oder leitet an eine vordefinierte Serviceroute weiter.

Abbildung 2: Vereinfachter Laufzeitablauf

Architektonisch wichtig ist hier der Hybrid-Ansatz: Agentische Logik mit klassischen NLU mit Entity-Recognition / Flow-Bausteinen zu kombinieren.

5. 5. Wie die Implementierung im Projekt aufgeteilt ist

5.1 Architect Bot Flow

Eigenschaft	Befund im Export
Flow-Name	GR Testhandel AVA v2
Sprachen	de-de; en-us
Virtual Agent	Im Flow aktiviert.
Klassische NLU	7 Intents und 7 Entities im Export.
Klassische Task-Logik	Eine Task „PLZ Zuordnung“ mit Normalisierung, Lookup und Bestätigungsfrage.
Knowledge im Flow	Produktwissen über Sharepoint Integration
Eskalation	Bei exitReason = Escalation erfolgt eine Weiterleitung an eine vorkonfigurierte Servicenummer.

Der YAML-Export zeigt, dass der Flow nicht nur Menüs abspielt. Er enthält Variablen, Regeln, Aufgaben, Sprachkonfiguration und den eigentlichen Übergabepunkt an den agentischen Agenten.

Aufbau

- Der Flow ist hybrid aufgebaut: klassische Intents/Slots/Tasks sind vorhanden, zugleich ist der agentische Agent der zentrale Laufzeitbaustein.
- Die Task „PLZ Zuordnung“ bereinigt die eingegebene PLZ, ergänzt bei Bedarf eine führende Null und sucht anschließend über eine Data Table nach dem passenden Markt.
- Zusätzliche klassische Loop-/Debug-Elemente sind im Export sichtbar. Für die Kernarchitektur dieser Doku ist vor allem der AVA-Aufruf mit Eskalationspfad relevant.

5.2 Agentic Virtual Agent in AI Studio

Aspekt	Zusammenfassung
Rolle	Der Agent tritt als „Lea“ im Kundencenter eines fiktiven Großhändlers auf und ist auf klar definierte Anliegen begrenzt.
Guidelines	Der Export enthält detaillierte Sprach- und Prozessregeln, z. B. Ziffer-für-Ziffer-Wiedergabe von IDs und Hinweise zum Verstehen gesprochener PLZ-Muster.
Guardrails	Es existieren inhaltliche Grenzen, z. B. keine Aussagen gegen Konkurrenz und keine Konkurrenzangaben.
Tools	Der Agent ist mit vier Tools verbunden: Knowledge (Sharepoint Connector), Lieferstatus, Markt-Lookup und Workitem-Anlage für Registrierung/Karte.
Struktur	Die Konfiguration arbeitet mit klaren Input-/Output-Typen und Ereignissen. Das macht Tool-Aufrufe nachvollziehbar und kontrollierbar.
Ausgabe an Architect	Architect erhält am Ende eine Exit-Information zurück und entscheidet dann über Gesprächsende oder Eskalation.

Angebundene Tools im Agentenexport

Tool	Typ	Zweck	Beispielhafte Daten
Produktinformation	Knowledge	FAQ-/Produktfragen beantworten.	Freitext-Frage -> Antwort aus Wissensbasis
get_shipment	Data Action	Lieferstatus zu einer Sendungsnummer abfragen.	ShipmentNumber HLCU1234567 -> Start, Ziel, ETA
Kartenbestellung	Data Action	Workitem für Registrierung/Kundenkarte anlegen.	Kundendaten -> Vorgangs-/Workitem-ID
Find next market based on location	Data Action	Nächsten Markt anhand der PLZ bestimmen.	PLZ -> Marktname, Stadt, Markt-PLZ

Besonders interessant ist, dass das Projekt zwei Wege für Markt-Lookups zeigt: einmal klassisch direkt im Bot Flow über Data Table Lookup und einmal agentisch als Tool. Das macht den hybriden Charakter der Lösung gut sichtbar.

6. 6. Daten, Integrationen und „Backend“-Bausteine

Baustein	Technische Funktion	Einsatz im Projekt
Data Table	Lokaler Lookup-Datensatz in Genesys Cloud	PLZ -> nächster Markt; klassische Nutzung in der Task „PLZ Zuordnung“.
CSV-Import	Versionierbarer Datenbestand	5.000 Zeilen und 4 Spalten für Marktzuordnungen; geeignet für Import in eine Genesys Data Table.
Knowledge	Wissensquelle für Antwortgenerierung	Produktinformationen / FAQ-orientierte Antworten im Agenten.
Data Actions	Integrationsschicht	Lieferstatus, Markt-Lookup und Workitem-Erstellung.
Workitem	Manuelle/Automatisierte Nachbearbeitung	Registrierungs- bzw. Kartenprozess wird als strukturierter Folgeauftrag angelegt.

7. 7. API-/SDK-Perspektive

Für eine technische Einordnung ist wichtig zu unterscheiden, wo hier „Code“ wirklich stattfindet. Im Repository ist keine eigenständige SDK-Anwendung enthalten. Die Lösung ist vor allem eine Plattform-Konfiguration mit Integrationspunkten.

Ebene	Im Projekt?	Bedeutung
No-code / UI	Ja	Architect und AI Studio bilden die Hauptimplementierung.
Data Actions / APIs	Ja	Der Agent greift über Tool-/Action-Definitionen auf strukturierte Daten und Prozesse zu.
Eigener Custom Service im Repo	Nein	Es gibt keinen separaten Backend-Service in diesem Repository.
Git-Versionierung	Ja	YAML-, JSON- und CSV-Exporte sind in GitHub nachvollziehbar versioniert.
Automatisierter Import/Deployment	Teilweise / optional	Architect-Flows können generell als YAML exportiert und wieder importiert werden; im Repo ist jedoch keine vollständige CI/CD-Pipeline enthalten.

Zusammenfassung: Das Projekt demonstriert weniger „klassische Softwareentwicklung im Backend“, sondern eher „konfigurierbare Orchestrierung plus integrierte Aktionen“ auf einer Contact-Center-Plattform.

8. 8. Grenzen des aktuellen Prototyps und sinnvolle Weiterentwicklung

• Die Eskalation geht aktuell auf eine zentral vorkonfigurierte Serviceroute. Eine spätere Version könnte dynamisch je Markt oder Anliegen weiterleiten.

• Im Export sind klassische Bot-Flow-Bestandteile und agentische Logik parallel sichtbar. Für eine produktive Übergabe wäre eine noch klarere Trennung oder Dokumentation dieses Hybrids sinnvoll.

• Das Repository zeigt die Konfiguration, aber nicht alle org-spezifischen Voraussetzungen wie OAuth-Clients, veröffentlichte Integrationen, Berechtigungen oder Zielsysteme.

• Für eine produktionsreife Lösung wären zusätzlich Monitoring, Testfälle, Fehlerbilder und ein reproduzierbarer Import-/Deployment-Prozess hilfreich.

9. 9. Verwendete Grundlagen und Referenzen

Projektartefakte

GitHub-Repository: https://github.com/georgyrudnev/Musterhandel-Agentic-Voicebot.

Offizielle Genesys-Dokumentation (Auswahl)

Architect overview / About Architect; Connect agentic virtual agents to Architect Virtual Agent flows; About AI Studio; About agentic virtual agents; Set up tools for an agentic virtual agent; About the Genesys Cloud data actions integration; Work with data tables / Data Table Lookup action / Import or export data tables; Knowledge workbench overview V2; Work with workitems; Define Architect flows using YAML.